

Supplementary Material

Decoupled Downsampling and Projection Experiments

In this alternative approach, the adapter performs downsampling and projection in decoupled fashion:

1. **Manual Downsampling:** We first apply a temporal downsampling operation, such as frame-stacking, to the speech encoder output H . This involves grouping H into non-overlapping blocks of k consecutive vectors, which are then flattened to form a new, shorter sequence $H' = \{h'_1, \dots, h'_{T''}\}$, where the new length is $T'' = \lfloor T'/k \rfloor$. Each new vector h'_t is defined as:

$$h'_t = \text{Flatten}(h_{k(t-1)+1}, \dots, h_{kt}) \in \mathbb{R}^{k \cdot d_{\text{speech}}}.$$

2. **Projection:** A separate projection-only network is then applied to each vector h'_t to map it from the downsampled feature space to the LLM's embedding dimension:

$$e_t = \text{Adapter}_{\text{projection}}(h'_t) \in \mathbb{R}^{d_{\text{LLM}}}.$$

The final output is the sequence of embeddings $E = \{e_1, \dots, e_{T''}\}$. Our proposed Multi-Resolution CNN model does **not** use this method. All other adapters use this method.

Indic-MR-HuBERT

In the experiments evaluating the *Speech Encoder Training Strategy Analysis* Hypothesis, the IMRH-MLP-Llama8B_{it} configuration contains an instance of MR-HuBERT (Shi et al. 2024) pretrained from scratch on the Train sets of IndicSUPERB (Javed et al. 2023) dataset for 40,000 steps. All other training configurations and hyperparameters remain unchanged as mentioned in the original paper.

Adapter Implementation Details

- **MLP-based adapters:** These adapters first downsample the speech encoder's output by grouping consecutive frames.

- **MLP (MLP):** A two-layer feedforward network. Given the downsampled input x' , the transformations are:

$$h = \text{ReLU}(\text{Linear}_{\text{in}}(x')), \quad y = \text{Linear}_{\text{out}}(h)$$

where $\text{Linear}_{\text{in}}$ projects from $\mathbb{R}^{d_{\text{speech}} \cdot k}$ to $\mathbb{R}^{2048}$, and $\text{Linear}_{\text{out}}$ maps to $\mathbb{R}^{d_{\text{LLM}}}$. This adapter is adopted as is from (Ma et al. 2025).

- **MLP-large ($AdaMLP_{\text{large}}$):** A more complex variant with a bottleneck architecture mapping the input from $\mathbb{R}^{d_{\text{speech}} \cdot k}$ through a $\mathbb{R}^{2048} \rightarrow \mathbb{R}^{1024} \rightarrow \mathbb{R}^{2048}$ sequence of layers, before a final projection to $\mathbb{R}^{d_{\text{LLM}}}$. A ReLU activation follows each intermediate transformation. This subtle change in the architecture yields in slight increase in number of parameters from 23.6M to 27.8M but leads to substantial improvement in the WER.

- **MLP-SLN (SLN):** This model refines the **MLP-large** architecture by replacing the ReLU activation with SiLU and adding a Layer Normalization step after each intermediate linear transformation. We experimented with 4 different size variants:

- * **Small (SLN_{small}):** The simplest variant, featuring a 2-layer MLP that maps the input from $\mathbb{R}^{d_{\text{speech}} \cdot k}$ through an intermediate $\mathbb{R}^{256}$ layer before the final projection to $\mathbb{R}^{d_{\text{LLM}}}$.
- * **Base (SLN):** A variant with a bottleneck architecture mapping the input from $\mathbb{R}^{d_{\text{speech}} \cdot k}$ through a $\mathbb{R}^{768} \rightarrow \mathbb{R}^{256} \rightarrow \mathbb{R}^{768}$ sequence of layers, before a final projection to $\mathbb{R}^{d_{\text{LLM}}}$.
- * **Large (SLN_{large}):** A more complex variant with a bottleneck architecture mapping the input from $\mathbb{R}^{d_{\text{speech}} \cdot k}$ through a $\mathbb{R}^{2048} \rightarrow \mathbb{R}^{1024} \rightarrow \mathbb{R}^{2048}$ sequence of layers, before a final projection to $\mathbb{R}^{d_{\text{LLM}}}$.
- * **X-Large (SLN_{xlarge}):** The largest variant, utilizing a bottleneck architecture that maps the input from $\mathbb{R}^{d_{\text{speech}} \cdot k}$ through a $\mathbb{R}^{3072} \rightarrow \mathbb{R}^{2048} \rightarrow \mathbb{R}^{3072}$ sequence of layers, before a final projection to $\mathbb{R}^{d_{\text{LLM}}}$.

- **Transformer-based adapters:** These adapters apply multi-layer transformer encoders to the downsampled speech features.

- **Base Transformer ($Trfm$):** A standard multi-layer transformer encoder composed of 4 layers, each with 12 attention heads, a hidden size of 768, and an intermediate dimension of 3072, utilizing GELU activation and a dropout rate of 0.1.
- **Conv-Based Transformer ($ConvTrfm$):** This version incorporates two 1D convolutional layers (stride 2, kernel size 3) between the second and third transformer layers, reducing the sequence length by a factor of 4. The transformer components maintain the same hyperparameters as the **Base Transformer**.
- **QFormer ($QFormer_{\text{xlarge}}$):** This adapter uses a set of 64 learnable queries to interact with speech features via cross-attention across 6 transformer layers. This maps the variable-length speech sequence to a fixed-length output. This variant of QFormer matches the exact configuration discussed in the (Li et al. 2023). 2 smaller versions, $QFormer_{\text{large}}$, and $QFormer$ are also trained.

- **Convolution-based adapters:** These adapters operate at the original sequence length without explicit downsampling.

- **Simple CNN ($AdaCNN$):** A three-layer CNN that uses stride-2 convolutions to downsample and transform features, following a $d_{\text{speech}} \rightarrow 1024 \rightarrow 1024 \rightarrow d_{\text{llm}}$ pipeline.
- **Multi-Resolution CNN ($AdaSpeech$):** This adapter uses parallel convolutional branches with varying kernel sizes. Each branch is followed by Batch Normalization, SiLU activation and the

outputs of each branch are concatenated. The concatenated tensor is then passed through another convolution1d layer followed by another set of Batch Normalization and SiLU activation. We explore four variants:

- * **Small** (*AdaSpeech_{small}*): 3 branches (kernels 3, 5, 7), 32 channels each.
- * **Base** (*AdaSpeech*): 4 branches (kernels 3, 5, 7, 9), 128 channels each.
- * **Large** (*AdaSpeech_{large}*): 5 branches (kernels 3, 5, 7, 9, 11), 256 channels each.
- * **XLarge** (*AdaSpeech_{xlarge}*): 6 branches (kernels 3, 5, 7, 9, 11, 13), 512 channels each.

Training Details

All models were trained on a single Nvidia H100 GPU with 256 GB RAM. Model optimization is achieved over 10 epochs with mixed precision using the AdamW optimizer (Loshchilov and Hutter 2019). We used a batch size of 6 with gradient accumulation over 2 steps. The learning rate was set to 10^{-4} with a linear warmup schedule for the first 1000 steps (increasing from 0 to $1e-4$), followed by a linear decay to 0 over the remaining training steps. For generation, we used a beam search of 4 and set {minimum length, top-p, repetition penalty, length penalty & temperature} to 1. No sampling was used.

References

- Javed, T.; Bhogale, K.; Raman, A.; Kumar, P.; Kunchukuttan, A.; and Khapra, M. M. 2023. Indic-SUPERB: A Speech Processing Universal Performance Benchmark for Indian Languages. In *AAAI Conference on Artificial Intelligence*, 12942–12950.
- Li, J.; Li, D.; Savarese, S.; and Hoi, S. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *International Conference on Machine Learning*, 19730–19742.
- Loshchilov, I.; and Hutter, F. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.
- Ma, Z.; Yang, G.; Yang, Y.; Gao, Z.; Wang, J.; Du, Z.; Yu, F.; Chen, Q.; Zheng, S.; Zhang, S.; and Chen, X. 2025. Speech Recognition Meets Large Language Model: Benchmarking, Models, and Exploration. In *AAAI Conference on Artificial Intelligence*, 24840–24848.
- Shi, J.; Inaguma, H.; Ma, X.; Kulikov, I.; and Sun, A. Y. 2024. Multi-resolution HuBERT: Multi-resolution Speech Self-Supervised Learning with Masked Unit Prediction. In *International Conference on Learning Representations (ICLR)*.